

Agenda

- Stream Programming
- Nested Classes
- Local Class

Stream operations

- Source of elements

```
String[] names = {"Smita", "Rahul", "Rachana", "Amit", "Shraddha", "Nilesh",  
"Rohan", "Pradnya", "Rohan", "Pooja", "Lalita"};
```

- Create Stream and display all names

```
Stream.of(names)  
    .forEach(s -> System.out.println(s));
```

- filter() -- Get all names ending with "a"
 - `Predicate<T>: (T) -> boolean`

```
Stream.of(names)  
    .filter(s -> s.endsWith("a"))  
    .forEach(s -> System.out.println(s));
```

- map() -- Convert all names into upper case
 - `Function<T,R>: (T) -> R`

```
Stream.of(names)  
    .map(s -> s.toUpperCase())  
    .forEach(s -> System.out.println(s));
```

- sorted() -- sort all names in ascending order
 - String class natural ordering is ascending order.
 - sorted() is a stateful operation (i.e. needs all element to sort).

```
Stream.of(names)  
    .sorted()  
    .forEach(s -> System.out.println(s));
```

- sorted() -- sort all names in descending order

- `Comparator<T>: (T,T) -> int`

```
Stream.of(names)
    .sorted((x,y) -> y.compareTo(x))
    .forEach(s -> System.out.println(s));
```

- `skip()` & `limit()` -- leave first 2 names and print next 4 names

```
Stream.of(names)
    .skip(2)
    .limit(4)
    .forEach(s -> System.out.println(s));
```

- `distinct()` -- remove duplicate names
 - duplicates are removed according to `equals()`.

```
Stream.of(names)
    .distinct()
    .forEach(s -> System.out.println(s));
```

- `count()` -- count number of names
 - terminal operation: returns long.

```
long cnt = Stream.of(names)
    .count();
System.out.println(cnt);
```

- `collect()` -- collects all stream elements into an collection (list, set, or map)

```
List<String> list = Stream.of(names)
    .collect(Collectors.toList());
// Collectors.toList() returns a Collector that can collect all stream
elements into a list
```

```
Set<String> set = Stream.of(names)
    .collect(Collectors.toSet());
// Collectors.toSet() returns a Collector that can collect all stream
elements into a set
```

- `reduce()` -- addition of 1 to 5 numbers

```
int result = Stream
    .iterate(1, i -> i+1)
    .limit(5)
    .reduce(0, (x,y) -> x + y);
```

- max() -- find the max string
 - terminal operation
 - See examples.

Collect Stream result

- Collecting stream result is terminal operation.
- Object[] toArray()
- R collect(Collector)
 - Collectors.toList(), Collectors.toSet(), Collectors.toCollection(), Collectors.joining()
 - Collectors.toMap(key, value)

Stream of primitive types

- Efficient in terms of storage and processing. No auto-boxing and unboxing is done.
- IntStream class
 - IntStream.of() or IntStream.range() or IntStream.rangeClosed() or Random.ints()
 - sum(), min(), max(), average(), summaryStatistics(),
 - OptionalInt reduce().

Optional<> type

- Few stream operations yield Optional<> value.
- Optional value is a wrapper/box for object of T type or no value.
- It is safer way to deal with null values.
- It mostly helps to avoid exceptions
- To create the optional object
 - opt = Optional.of("A")
 - opt = Optional.empty() -> creates an optional with no value
- Get value from the Optional<>:
 - optValue = opt.get(); // if you know value exists
 - optValue = opt.orElse(defValue); // if you dont know value is present or not
- Consuming Optional<> value:
 - opt.isPresent() --> boolean;
 - opt.ifPresent(consumer);

Member/Nested classes

- By default all Java classes are top-level.
- In Java, classes can be written inside another class/method. They are Member classes.
- Four types of member/nested classes
 - Static member classes --
 - Non-static member class --
 - Local class --
 - Anonymous Inner class --
- When .java file is compiled, separate .class file created for outer class as well as inner class.

Static member classes

- Like other static members of the class (belong to the class, not the object).
- Accessed using outer class (Doesn't need the object of outer class).
- Can access static (private/public) members of the outer class directly.
- Static member class cannot access non-static members of outer class directly.
- The outer class can access all members (including private) of inner class directly (no need of getter/setter).
- The static member classes can be private, public, protected, or default.

```
class Outer {
    private int nonStaticField = 10;
    private static int staticField = 20;

    public static class Inner {
        public void display() {
            System.out.println("Outer.nonStaticField = " + nonStaticField);
            // error
            System.out.println("Outer.staticField = " + staticField); // ok
        }
    }
}

- 20

public class Main {
    public static void main(String[] args) {
        Outer.Inner obj = new Outer.Inner();
        obj.display();
    }
}
```

Non-static member classes/Inner classes

- Like other non-static members of the class (belong to the object/instance of Outer class).

- Accessed using outer class object (Object of outer class is MUST).
- Can access static & non-static (private) members of the outer class directly.
- The outer class can access all members (including private) of inner class directly (no need of getter/setter).
- The non-static member classes can be private, public, protected, or default.

```

class Outer {
    private int nonStaticField = 10;
    private static int staticField = 20;
    public class Inner {
        public void display() {
            System.out.println("Outer.nonStaticField = " + nonStaticField);
            System.out.println("Outer.staticField = " + staticField); // ok-
        }
    }
}

// ok-10
20

public class Main {
    public static void main(String[] args) {
        //Outer.Inner obj = new Outer.Inner(); // compiler error
        // create object of inner class
        //Outer outObj = new Outer();
        //Outer.Inner obj = outObj.new Inner();
        Outer.Inner obj = new Outer().new Inner();
        obj.display();
    }
}

```

- If Inner class member has same name as of outer class member, it shadows (hides) the outer class member. Such Outer class members can be accessed explicitly using `Outer.this`.

Static member class and Non-static member class

-- Application

```

// top-level class
class LinkedList {
    // static member class
    static class Node {
        private int data;
        private Node next;
        // ...
    }
    private Node head;
    // non-static member class
    class Iterator {

```

```

        private Node trav;
        // ...
    }
    // ...
    public void display() {
        Node trav = head;
        while(trav != null) {
            System.out.println(trav.data);
            trav = trav.next;
        }
    }
}

```

Local class

- Like local variables of a method.
- The class scope is limited to the enclosing method.
- If enclosed in static method, behaves like static member class. If enclosed in non-static method, behaves like non-static member class.
- Along with Outer class members, it can also access (effectively) final local variables of the enclosing method.
- We can create any number of objects of local classes within the enclosing method.

```

public class Main {
    private int nonStaticField = 10;
    private static int staticField = 20;
    public static void main(String[] args) {
        final int localVar1 = 1;
        int localVar2 = 2;
        int localVar3 = 3;
        localVar3++;
        // local class (in static method) -- behave like static member class
        class Inner {
            public void display() {
                System.out.println("Outer.nonStaticField = " +
nonStaticField); // error
                System.out.println("Outer.staticField = " + staticField); //
ok 20
                System.out.println("Main.localVar1 = " + localVar1); // ok 1
                System.out.println("Main.localVar2 = " + localVar2); // ok 2
                System.out.println("Main.localVar3 = " + localVar3); //
error
            }
        }
        Inner obj = new Inner();
        obj.display();
        //new Inner().display();
    }
}

```

Anonymous Inner class

- Creates a new class inherited from the given class/interface and its object is created.
- If in static context, behaves like static member class. If in non-static context, behaves like non-static member class.
- Along with Outer class members, it can also access (effectively) final local variables of the enclosing method.

```
// (named) local class
class EmpnoComparator implements Comparator<Employee> {
    public int compare(Employee e1, Employee e2) {
        return e1.getEmpno() - e2.getEmpno();
    }
}
Arrays.sort(arr, new EmpnoComparator());    // anonymous obj of local class
```

```
// Anonymous inner class
Comparator<Employee> cmp = new Comparator<Employee>() {
    public int compare(Employee e1, Employee e2) {
        return e1.getEmpno() - e2.getEmpno();
    }
};
Arrays.sort(arr, cmp);
```